

第 13 章 常见错误分析

C 语言的功能强,使用方便灵活,所以得到广泛的使用,它使程序设计人员有发挥聪明才智、显示编程技巧的机会。一个有经验的 C 程序设计人员可以编写出能解决复杂问题的、运行效率高的、占内存少的高质量程序。

但是要真正学好 C,用好 C,并不容易,“灵活”固然是好事,但也使人难以掌握,有的初学者往往出了错还不知怎么回事。C 编译程序对语法的检查不如其他高级语言那样严格。因此,往往要由程序设计者自己设法保证程序的正确性。调试一个 C 程序要比调试一个 Pascal 或 FORTRAN 程序更困难一些,需要不断积累经验,提高程序设计和调试程序的水平。

下面将初学者在学习和使用 C 语言时容易犯的错误列举出来,以起提醒的作用。这些内容在以前各章中大多已谈到,为便于查阅,在本章集中列举,以提醒初学者,以此为鉴。

(1) 忘记定义变量。

例如:

```
int main()
{
    x=3;
    y=6;
    printf("%d\n",x+y);
    return 0;
}
```

C 要求对程序中用到的每一个变量都必须定义其类型,上面程序中没有对 x,y 进行定义。应在函数体的开头加

```
int x,y;
```

(2) 输入输出的数据的类型与用户指定的输入输出格式声明不一致。例如,若 a 已定义为整型,b 已定义为实型:

```
int a=6;
float b=4.5;
printf("%f,%d\n",a,b);
```

编译时不给出出错信息,但运行结果将与原意不符,输出为

```
0.000000,1074921472
```

这显然是错误的。原因是数据类型与指定的输出格式不匹配。在这种情况下,并不是按照赋值的规则进行转换(如把 b 的值 4.5 转换成 4,然后输出 4),而是将数据在存储单元中的形式按格式符的要求组织输出。如 a 是整数,按整数的存储方式存储,今要它按浮点数输

出,系统把此数在内存中存放的形式按浮点数解释,组织输出。b 是浮点数,按浮点数的存储方式存储,现在系统把这个数在内存中存放的形式按整数解释,把它直接作为某一整数输出。

(3) 未注意 int 和 short 数据的数值范围。Turbo C 等编译系统,对一个 int 和 short 型数据分配 2 个字节。因此一个整数的范围为 $-2^{15} \sim 2^{15} - 1$, 即 $-32768 \sim 32767$ 。常见这样的程序段:

```
int num;
num=89101;
printf("%d",num);
```

如果用 Turbo C 编译系统,得到的却是 23565,原因是 90101 已超过 32767。2 个字节容纳不下 89101,则将高位截去(见图 13.1),即将超过低 16 位的数截去,也即将 89101 减去 2^{16} (即 16 位二进制所形成的模): $89101 - 65535 = 23565$ 。

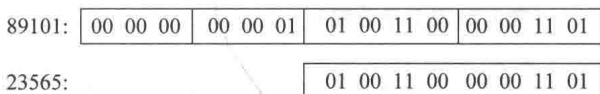
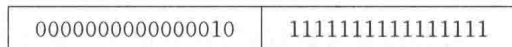


图 13.1

有时还会出现负数。例如:

```
num=198607;
```

输出得-1。因为 198607 的二进制形式为



只把后面两个字节(低 16 位)中的值存放在 num 中,而它是一 1 的补码。因此输出一 1。

对于超过整数范围的数,要用 long 型,即改为

```
long int num;
num=89101;                                (long 占 4 个字节,能存放 89101)
```

如果用以下输出:

```
printf ("%d",num);
```

在输出时仍用“%d”说明符,也会出现以上错误。对 long 型整数,应该用“%ld”格式输出。

如果用 Visual C++, 一个整数占 4 个字节,能容纳上面的数据,不会出错。但它也是有范围的,无非允许的范围比较大而已。如果在 Visual C++ 用 short 型变量来容纳前面的数,也会出现同样的问题。

(4) 在使用输入函数 scanf 时,忘记用变量的地址符 &。

例如:

```
scanf("%d%d",a,b);
```

这是许多初学者刚学习 C 语言时一个常见的疏忽,在其他语言中在输入时只须写出变量名即可,而 C 语言要求指明“向哪个地址所标识的单元送值”。应写成

```
scanf("%d%d",&a,&b);
```

(5) 输入数据的形式与要求不符。

用 scanf 函数输入数据,应注意如何组织输入数据。假如有以下 scanf 函数:

```
scanf("%d%d",&a,&b);
```

有人按下面的方法输入数据:

```
3,4 (数据间用逗号分隔)
```

这是错的。数据间应该用空格(或 Tab 键,或回车符)来分隔。读者可以用

```
printf("%d %d",a,b);
```

来验证一下。

根据上面的 scanf 函数的写法,应当用以下方法输入:

```
3 4 (数据间用空格分隔)
```

如果 scanf 函数形式为

```
scanf("%d,%d",&a,&b);
```

对 scanf 函数中格式字符串中除了格式说明符外,对其他字符必须按原样输入。因此,应按以下方法输入:

```
3,4
```

此时如果用“3 4”反而错了。

(6) 在用 scanf 函数向字符数组输入数据时,在数组名前面多加了 &。例如:

```
char a[20];
scanf("%s",&a);
```

在对变量输入数据时应当加 &,比如 &a,这是变量 a 的地址,表示“数据送到这个地址去,放在其标志的存储单元中”。而现在数组名 a 本身就是地址,再加 & 就是画蛇添足了。只须直接写数组名即可:

```
scanf("%s",a);
```

这样,输入的字符串就会送到数组名 a 所代表的地址去,存放在以此地址开头的一段存储单元中。

(7) 在用 scanf 函数向数值型数组输入数据时,用数值型数组名。例如:

```
int a[20];
scanf("%d",a);
```

这是错误的。对字符数组,可以在 scanf 函数中通过格式符 %s 和数组名输入一个字符串,但对数值型数组不能照搬。因为输入的不是一个数据,而是多个数据,必须分别通过指定数组元素输入。即:

```
int a[20];
int i;
```

```
for (i=0;i<20;i++)  
    scanf("%d",&a[i]);
```

(8) 语句后面漏分号。

C 语言规定语句末尾必须有分号。分号是 C 语句不可缺少的一部分。这也是和其他语言不同的。有的初学者往往忘记写这一分号。例如：

```
a=3  
b=4;
```

在程序编译时,编译系统在“a=3”后面未发现分号,就接着检查下一行有无分号。“b=4”也作为上一行的语句的一部分,这就出现语法错误。由于在第 2 行才能判断语句有错,所以编译系统指出“在第 2 行有错”,但用户在第 2 行却未发现错误。这时应该检查上一行是否漏了分号。

(9) 把预处理指令当作 C 语句,在行末加了分号。如:

```
#include <stdio.h>;
```

预处理指令(包括常用的 #include、#define 指令)不是 C 语句,在指令后面不应加分号。预处理指令不能被编译,必须先由预处理器对之加工处理,成为能被编译系统识别和编译的 C 程序,才真正进行编译。

(10) 在不该加分号的地方加了分号。

例如:

```
if (a>b);  
    printf("a is larger than b\n");
```

本意是:当 $a>b$ 时输出“a is larger than b”的信息。由于在 $\text{if}(a>b)$ 后加了分号,if 语句到分号结束。即当 $a>b$ 为真时,执行一个空语句。本来想 $a\leq b$ 时不输出上述信息,但现在 printf 函数语句并不从属于 if 语句,而是与 if 语句平行的语句。不论 $a>b$ 还是 $a\leq b$,都输出“a is larger than b”。

又如:

```
for(i=0;i<10;i++);  
{scanf("%d",&x);  
  printf("%d\n",x*x);  
}
```

本意为先后输入 10 个数,每输入一个数后输出它的平方值。由于在 $\text{for}(i=0;i<10;i++)$ 后不经意地加了一个分号,使循环体变成了空语句。执行 for 语句的效果只是使变量 i 的值由 0 变到 10。然后输入一个整数并输出它的平方值。这种错误往往发生在不熟悉 C 语法的初学者身上。

总之,在 if,for,while 等语句中,不要乱加分号,以免画蛇添足。

(11) 对应该有花括号的复合语句,忘记加花括号。例如:

```
sum=0;  
i=1;
```

```
while (i<=100)
    sum=sum+i;
    i++;
```

本意是想实现 $1+2+\cdots+100$, 即 $\sum_{i=0}^{100} i$, 但上面的语句只是重复了 $\text{sum}+i$ 的操作, 而且循环永不终止, 因为 i 的值始终没有改变。错误在于没有写成复合语句形式, 造成 `while` 语句的范围到其后第 1 个分号为止, 而语句“`i++;`”不属于循环体范围之内。应改为

```
sum=0;
i=1;
while (i<=100)
{
    sum=sum+i;
    i++;
}
```

(12) 括号不配对。

当一个语句中使用多层括号时常出现这类错误, 纯属粗心所致。例如:

```
while((c=getchar())!='#')
    putchar(c);
```

少了一个右括号。

(13) 在用标识符时, 混淆了大写字母和小写字母的区别。

例如:

```
int main()
{
    int A,B,C;
    a=2;b=3;
    c=a+b;
    printf("%d+%d=%d\n",a,b,c);
    return 0;
}
```

编译时出错。编译程序把 a 和 A 认作两个不同的变量名处理, 同样 b 和 B , c 和 C 都分别代表两个不同的变量。所以认为“变量 a, b, c 未经定义”, 出错。

(14) 误把“`=`”作为“等于”运算符。

许多人习惯性地用数学上的等于号“`=`”用作 C 关系运算符“等于”。而在 C 语言中, “`==`”才是关系运算符“等于”。有人写出如下的 `if` 语句:

```
if (score=100) n++;
```

本意是想统计 score 为 100 分的人数, 当 score 等于 100 时就使 n 加 1。但 C 编译系统将“`=`”作为赋值运算符, 将“`score=100`”作为赋值表达式处理, 把 100 赋给 score , 作为 score 的新值。if 语句检查 score 是否为零。若为非零, 则作为“真”; 若为零作为“假”。今 score 经过赋值之后显然不等于 0, 因此总执行 `n++`, 不论 score 的原值是什么, 都使 n 的值加 1。

这种错误在编译时是检查不出来的, 但运行结果往往是错的。而且由于习惯的影响, 在

检查源程序时,往往设计者自己是不易发觉的。

(15) 引用数组元素时误用了圆括号。

例如:

```
int main()
{int i,a(10);                      //应写为 a[10]
  for(i=0;i<10;i++)
    scanf("%d",&a(i));           //应写为 &a[i]
  :
}
```

C 语言中对数组的定义或引用数组元素时必须用方括号。

(16) 在定义数组时,将定义的“元素个数”误认为是“可使用的最大下标值”。

例如:

```
int main()
{int a[10]={1,2,3,4,5,6,7,8,9,10};
  int i;
  for(i=1;i<=10;i++)
    printf("%d",a[i]);
  return 0;
}
```

编程者想输出 $a[1] \sim a[10]$ 这 10 个元素。这是不可能的。C 语言规定在定义数组时用 $a[10]$, 表示 a 数组有 10 个元素, 而不是可以用的最大下标值为 10。在 C 语言中数组的下标是从 0 开始的, 因此, 数组 a 只包括 $a[0] \sim a[9]$ 这 10 个元素, 想引用 $a[10]$, 就超出 a 数组的范围了。值得注意的是, 在程序编译时, C 编译系统对此并不报错, 编译能通过, 但运行结果不对。系统把 $a[9]$ 后面的存储单元作为 $a[10]$ 输出, 这显然不是编程者的原意。由于编译系统不报错, 有时编程者难以发现这类错误。要注意仔细分析运行结果。

(17) 对二维或多维数组的定义和引用的方法不对。

例如:

```
int main()
{int a[5,4];
  printf("%d",a[1+2,2+2]);
  return 0;
}
```

这是把数学上的用法习惯性地用于 C 程序中。在 C 语言中规定: 二维数组和多维数组在定义和引用时必须将每一维的数据分别用方括号括起来。上面 $a[5,4]$ 应改为 $a[5][4]$, $a[1+2,2+2]$ 应改为 $a[1+2][2+2]$ 。根据 C 的语法规则, 在一个方括号中的是一个维的下标表达式, 系统把 $a[1+2,2+2]$ 方括号中的“ $1+2,2+2$ ”作为一个逗号表达式处理, 它的值是第 2 个数值表达式的值, 即 $2+2$ 的值, 即 4。所以 $a[1+2,2+2]$ 相当于 $a[4]$, 而 $a[4]$ 是 a 数组的第 4 行的首地址。因此执行 printf 函数输出的结果并不是 $a[3][4]$ 的值, 而是 a 数组第 4 行的首地址。

(18) 误以为数组名代表数组中全部元素。

例如:

```
int main()
{int a[4]={1,3,5,7},b[4];
  b=a;;
  return 0;
}
```

企图把 a 数组的全部元素的值赋给 b 数组相应的元素,这是错误的。在 C 语言中,数组名 a 代表数组 a 的首元素地址,数组名 b 代表数组 b 的首元素地址,二者均是地址常量,不能被赋值。

(19) 混淆字符数组与字符指针的区别。

例如:

```
int main()
{char str[14];
  str="Computer and C";
  printf("%s\n",str);
  return 0;
}
```

编译出错。str 是数组名,代表数组首地址。在编译时对 str 数组分配了一段内存单元,因此在程序运行期间 str 是一个常量,不能再被赋值。所以

```
str="Computer and C";
```

是错误的。

如果把

```
char str[14];
```

改成

```
char * str;
```

则程序正确。此时 str 是指向字符数据的指针变量,“str="Computer and C";”是合法的,它将字符串的首地址赋给指针变量 str,然后在 printf 函数语句中输出字符串"Computer and C"。

(20) 在引用指针变量之前没有对它赋予确定的值。例如:

```
int main()
{char * p;
  scanf("%s",p);
  :
}
```

没有给指针变量 p 赋值就引用它,编译时给出警告信息。其实指针变量 p 中不是空的,而是存放了一个不可预测的值,p 指向一个地址为 p 的存储单元,而这个存储单元中可能是存放了有用的数据的。如果执行上面的 scanf 语句,就将一个字符串输入到此存储单元开始的

一段存储空间,这就改变了这段存储空间的原有状况,有可能破坏了系统的工作环境,产生灾难性的后果,十分危险。应当改为

```
int main()
{
    char * p, c[20];
    p = c;
    scanf("%s", p);
    :
}
```

即先根据需要定义一个大小合适的字符数组 c,然后将 c 数组的首地址赋给指针变量 p,此时 p 有确定的值,指向数组 c 的首元素。再执行 scanf 函数就没有问题了,把从键盘输入的字符串存放到字符数组 c 中。

(21) switch 语句的各分支中漏写 break 语句。

例如:

```
switch(score)
{
    case 5: printf("Very good!");
    case 4: printf("Good!");
    case 3: printf("Pass!");
    case 2: printf("Fail!");
    default: printf("data error!");
}
```

写上述 switch 语句的原意是希望根据 score(成绩)输出评语。但当 score 的值为 5 时,输出为

Very good! Good! Pass! Fail! data error!

原因是漏写了 break 语句。case 只起标签的作用,而不起判断作用,因此在执行完第 1 个 printf 函数语句后接着执行第 2,3,4,5 个 printf 函数语句。应改为

```
switch(score)
{
    case 5: printf("Very good!");break;
    case 4: printf("Good!");break;
    case 3: printf("Pass!");break;
    case 2: printf("Fail!");break;
    default: printf("data error!");break;
}
```

(22) 混淆字符和字符串的表示形式。

例如:

```
char sex;
sex = "M";
:
```

sex 是字符变量,只能存放一个字符。而字符常量的形式是用单撇号括起来的,"M"是用双

撇号括起来的字符串,它包括两个字符: 'M' 和 '\0', 无法存放到字符变量 sex 中。应改为

```
sex='M';
```

(23) 使用自加(++)和自减(--)运算符时容易出的错误。

例如:

```
int main()
{
    int *p, a[5] = {1, 3, 5, 7, 9};
    p = a;
    printf("%d", *p++);
    return 0;
}
```

有的人认为“*p++”的作用是先使 p 加 1, 即指向第 1 个元素 a[1] 处, 然后输出第 1 个元素 a[1] 的值 3。其实不然, 由于++的优先级高于* (见附录 C), 因此先执行 p++, 而 p++ 的作用是先使用 p 的原值进行运算 (进行 *p 的运算), 然后再使 p 加 1。p 原来指向数组 a 的第 0 个元素 a[0], 因此 *p 就是第 0 个元素 a[0] 的值 1。结论是先输出 a[0] 的值, 然后再使 p 加 1。如果是 *(++p), 则先使 p 指向 a[1], 然后输出 a[1] 的值。

在使用++和--运算符时, 一定要避免歧义性, 如无把握, 宁可多加括号。如上面的 *p++ 可改写为 *(p++)。

(24) 忘记对所调用的函数进行函数原型声明。

例如:

```
int main()
{
    float a, b, c;
    a = 3.5; b = -7.6;
    c = max(a, b);
    printf("%f\n", c);
    return 0;
}

float max(float x, float y)
{
    return (z = x > y ? x : y);
}
```

这个程序乍看起来没有什么问题, 但在编译时有出错信息。原因是 max 函数是在 main 函数之后才定义, 也就是 max 函数的定义位置在 main 函数调用 max 函数之后。编译系统在编译到“c=max(a,b);”时, 无法判定 max 是什么, 无法处理。方法可以用以下二者之一:

① 在 main 函数中增加一个对 max 函数的声明, 即函数的原型声明:

```
int main()
{
    float max(float x, float y);           //对 max 函数的声明
    float a, b, c;
    a = 3.5; b = -7.6;
    c = max(a, b);
}
```

```
printf("%f\n",c);
return 0;
}
```

② 将 max 函数的定义位置调到 main 函数之前,即:

```
float max(float x,float y)
{
    return(z=x>y? x:y);
}

int main()
{float a,b,c;
  a=3.5;b=-7.6;
  c=max(a,b);
  printf("%f\n",c);
  return 0;
}
```

这样,由于 max 函数的定义出现在对它的调用之前,所以编译时不会出错,程序运行结果是正确的。但是,提倡用第①种方法,符合规范,应养成对函数都作函数声明的习惯。

(25) 对函数声明与函数定义不匹配。

如已定义一个 fun 函数,其首行为

```
int fun(int x,float y,long z)
```

在主调函数中作下面的声明将出错。

fun(int x,float y,long z);	//漏写函数类型
float fun(int x,float y,long z);	//函数类型不匹配
int fun(int x,int y,int z);	//参数类型不匹配
int fun (int x,float y);	//参数数目不匹配
int fun(int x,long z,float y);	//参数顺序不匹配

下面的声明是正确的:

int fun(int x,float y,long z);	
int fun(int,float,long);	//可以不写形参名
int fun(int a,float b,long c);	//编译时不检查函数原型中的形参名

(26) 在需要加头文件时没有用 #include 指令去包含头文件。

例如: 程序中用到 fabs 函数,没有用 #include<math.h>,程序中用到输入输出函数,没有用 #include<stdio.h>,等等。

这是不少初学者常犯的错误。至于哪个函数应该用哪个头文件,可参阅教材附录 E。

(27) 误认为函数形参值的改变会影响实参的值。

例如:

```
int main()
{int a,b;
```

```

a=3;b=4;
swap(a,b);
printf("%d,%d\n",a,b);
return 0;
}

```

```

void swap(int x,int y)
{int t;
 t=x;x=y;y=t;
}

```

编程者原意是通过调用 swap 函数使 a 和 b 的值对换,然后在 main 函数中输出已对换过值的 a 和 b。但是这样的程序是达不到目的的,因为 x 和 y 的值的改变是不传回实参 a 和 b 的,main 函数中的 a 和 b 的值并未改变。

如果想从函数得到一个以上的变化了的值,应该用指针变量。用指针变量作函数参数,使指针变量所指向的变量的值发生变化。此时变量的值改变了,主调函数中可以利用这些已改变的值。例如:

```

int main()
{int a,b,*p1,*p2;
 a=3;b=4;
 p1=&a;p2=&b;
 swap(p1,p2);
 printf("%d,%d\n",a,b);           //a 和 b 的值已对换
 return 0;
}

void swap(int *pt1,int *pt2)
{int temp;
 temp=*pt1;*pt1=*pt2;*pt2=temp;
}

```

(28) 函数的实参和形参类型不一致。例如:

```

#include <stdio.h>
int main()
{int fun(int x,int y);
 float a=3.5,b=4.6,c;
 c=fun(a,b);                       //实参 a 和 b 为 float 型
 printf("%f\n",c);
 return 0;
}

int fun(int x,int y)                //形参 x 和 y 为 int 型
{
 return x+y;
}

```

实参 a 和 b 为 float 型,形参 x 和 y 为 int 型。在编译时,系统给出“警告”。a 和 b 的值传递给 x 和 y 时,会按赋值规则处理,把小数部分删去,这样得不到应有的运行结果。应当做到实参与形参的类型一致。

(29) 不同类型的指针混用。例如:

```
int main()
{
    int i=3, * p1;
    float a=1.5, * p2;
    p1=&i;           //p1 指向整型变量 i
    p2=&a;           //p2 指向 float 型变量 a
    p2=p1;          //把 p1 的值赋给 p2
    printf("%d,%d\n", * p1, * p2);
    return 0;
}
```

企图使 p2 也指向 i,但 p2 是指向 float 型变量的指针,不能指向整型变量。编译时出现“警告”,如果运行,结果是:

3,0

显然结果不对。指向不同类型的指针间的赋值必须进行类型转换。例如:

```
p2=(float *)p1;
```

作用是先将 p1 的值转换成指向实型的指针,然后再赋给 p2。有的编译系统会自动进行类型转换。转换后,p2 获得 p1 的值(是一地址),但 p2 仍然只能指向 float 型数据。由于指针类型不同,无法通过 * p2 得到 i 的整型值。

类型转换在 C 程序中是常见的。例如,用 malloc 函数开辟内存单元,函数返回的是指向被分配内存空间的 void * 类型的指针。而人们希望开辟的是存放一个结构体变量值的存储单元,要求得到指向该结构体变量的指针,可以进行如下的类型转换:

```
struct student
{
    int num;
    char name[20];
    float score;
};

struct student student1, * p;
p=(struct student *)malloc(LEN);
```

p 是指向 struct student 结构体类型数据的指针,将 malloc 函数返回的 void * 类型指针转换成指向 struct student 类型变量的指针。这个情况与上面的情况有所不同,指针变量 p 在定义时已确定它是指向 struct student 类型变量的,它获得了用 malloc 函数开辟的空间的起始地址。在这个空间内存放 struct student 类型的数据,通过 p 来引用这些数据,这是合法的、常用的方法。

(30) 没有注意系统对函数参数的求值顺序的处理方法。

例如,有以下语句:

```
int i=3;
printf("%d,%d,%d\n",i,++i,++i);
```

许多人认为输出必然是

3,4,5

其实不尽然。在 Turbo C 和 Visual C++ 6.0 系统中输出是

5,5,4

因为这些系统的处理方法是：按自右至左的顺序求函数参数的值。先求出最右面一个参数(++i)的值为 4,再求出第 2 个参数(++i)的值为 5,最后求出最左面的参数(i)的值 5。

如果改为下面的 printf 语句：

```
printf("%d,%d,%d\n",i,i++,i++);
```

在 Turbo C 和 Visual C++ 6.0 系统中输出是

3,3,3

求值的顺序仍然是自右而左,但是需要注意的是：对于 i++,什么时候执行 i 自加 1 的操作？由于 i++是“后自加”,是在执行完 printf 语句后再使 i 加 1,而不是在求出最右面一项的值(值为 3)之后 i 的值立即加 1,所以 3 个输出项的值都是 i 的原值 3。

C 标准没有具体规定函数参数求值的顺序是自左至右,还是自右至左。但每个 C 编译程序都规定了自己的顺序,在多数情况下,从左到右求解和从右到左求解的结果是相同的,所以人们对此没有感到有什么问题。例如：

```
fun1(a+b,b+c,c+a);
```

fun1 是一个函数名,有 3 个实参表达式：a+b、b+c、c+a。在一般情况下,自左至右地求这 3 个表达式的值和自右至左地求它们的值是一样的,但前面举的例子是不相同的。因此,应该使程序具有通用性,不会在不同的编译环境下得到不同的结果。不使用会引起二义性的用法。如果在上例中,希望输出“3,4,5”时,可以改用

```
i=3; j=++i;k=++j;
printf("%d,%d,%d\n",i,j,k);
```

(31) 混淆数组名与指针变量的区别。

例如：

```
int main()
{int i,a[5];
  for(i=0;i<5;i++)
    scanf("%d",&a[i]);
  return 0;
}
```

企图通过改变 a 的值使指针下移,每次指向下一个数组元素。它的错误在于不了解数组名代表数组首地址,它的值是不能改变的,用 a++是错误的,应当用指针变量来指向各数组

元素,即

```
int i,a[5],*p;;
p=a;
for(i=0;i<5;i++)
    scanf("%d",p++);
```

或

```
int a[5],*p;;
p=a;
for(p=a;p<a+5;p++)
    scanf("%d",p);
```

(32) 混淆结构体类型与结构体变量的区别,对一个结构体类型赋值。

例如:

```
struct worker
{
    int num;
    char name[10];
    char sex;
    int age;
};
worker.num=187045;
strcpy(worker.name,"Zhang Fang");
worker.sex='M';
worker.age=18;
```

这是错误的,struct worker 是类型名,不是变量,不占存储单元。只能对结构体变量中的成员赋值,不能对类型中的成员赋值。上面的程序段未定义变量,应改为

```
struct worker
{
    int num;
    char name[10];
    char sex;
    int age;
};
struct worker worker_1; //定义结构体变量 worker_1
worker_1.num=187045;
strcpy(worker_1.name,"Zhang Fang");
worker_1.sex='M';
worker_1.age=18;
```

今定义了结构体变量 worker_1,并对其中的各成员赋值,这是合法的。

(33) 使用文件时忘记打开,或打开方式与使用情况不匹配。

例如,用只读方式打开文件,却企图向该文件输出数据,例如:

```
if ((fp=fopen("test","r"))==NULL)
```

```
{printf("cannot open this file\n");
  exit(0);
}
ch=fgetc(fp);
while(ch!='#')
{ch=ch+4;
  fputc(ch,fp);
  ch=fgetc(fp);
}
```

对以 r 方式(只读方式)打开的文件,进行既读又写的操作,显然是不行的。

(34) 在打开文件时,指定的文件名找不到。

如:

```
if ((fp=fopen("test", "r")) == NULL)
{printf("cannot open this file\n");
  exit(0);
}
```

想打开一个已存在的文件 test,以便从中读数据,但是并不存在这个文件,或者这个文件不在用户当前目录下,系统找不到。如果想打开的文件不在用户当前目录下,应当在指定文件名时指出文件路径。如:

```
if ((fp=fopen("D:\temp\test", "r")) == NULL)
{printf("cannot open this file\n");
  exit(0);
}
```

(35) 忘记关闭文件,虽然系统会自动关闭所用文件,但可能会丢失数据。因此必须在用完文件后关闭它。

以上只是列举了一些初学者常出现的错误,这些错误大多是由于 C 语法不熟悉之故。对 C 语言使用多了,比较熟练了,犯这些错误自然就会减少了。在深入使用 C 语言后,还会出现其他一些更深入、更隐蔽的错误。